Prinsip SOLID Bagian 2 — ISP, DIP, dan Keterkaitan Kelima Prinsip

Mata Kuliah:
CIF61123 – Pola Perancangan (3 SKS)
Pekan ke-5 | Sub-CPMK13
Disusun oleh:
Kelompok Pengelola Mata Kuliah Pola Perancangan



Versi Dokumen: 1.0
Tanggal: 17; Bulan: Agustus; Tahun: 2026
© Fakultas Ilmu Komputer, Universitas Brawijaya

filkom.ub.ac.id

Peta Perkuliahan

Kedudukan pertemuan ini di antara enam belas pekan perkuliahan

01	Konsep dasar & klasifikasi pola
02	Prinsip perancangan berorientasi objek
03	Interface, coupling–cohesion, UML
04	SOLID bagian 1 — SRP, OCP, LSP
05	**SOLID bagian 2 — ISP, DIP**
06	Code smell & refactoring
07	Klaster 1 — kontrol pembuatan objek
08	Ujian Tengah Semester
09	Klaster 2 — komposisi rekursif
10	Klaster 3 — pembungkus antarmuka
11	Klaster 4 — algoritma bervariasi
12	Klaster 5 — ketergantungan longgar
13	Katalog GoF & pedoman pemilihan
14	Studi kasus terpadu bagian 1
15	Studi kasus terpadu bagian 2
16	Ujian Akhir Semester

© Fakultas Ilmu Komputer, Universitas Brawijaya

SubCPMK yang Didukung

Pertemuan kedua dari tiga pertemuan untuk Sub-CPMK13

13 Sub-CPMK13

Mampu menganalisis penerapan prinsip SOLID dan melakukan refactoring pada kode program Java.

Lanjutan dari: pekan ke-4 yang membahas SRP, OCP, dan LSP.

F Bentuk penilaian pekan ini

Formatif — latihan analisis kode dan diskusi di kelas.

Bobot: 0%.

Menjadi bekal langsung bagi LK02 pada pekan ke-6.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Materi Pembelajaran

Empat pokok bahasan, bersumber dari [2] dan P3

1. **Interface Segregation**

   Antarmuka kecil dan sesuai kebutuhan tiap klien.

2. **Dependency Inversion**

   Aturan bisnis tidak boleh bergantung pada rincian teknis.

3. **Keterkaitan kelima prinsip**

   Bagaimana SOLID bekerja sebagai satu kesatuan.

4. **Studi kasus rancangan berlapis**

   Penerapan SOLID pada arsitektur tiga lapis.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Tujuan Pembelajaran

Setelah pertemuan ini mahasiswa diharapkan mampu:

- Menjelaskan dan menerapkan Interface Segregation Principle pada rancangan antarmuka.
- Menjelaskan dan menerapkan Dependency Inversion Principle beserta mekanisme dependency injection.
- Membedakan arah ketergantungan sebelum dan sesudah inversi diterapkan.
- Menjelaskan keterkaitan antarkelima prinsip SOLID sebagai satu kesatuan.
- Menganalisis penerapan SOLID pada rancangan berlapis dan mengusulkan perbaikannya.

© Fakultas Ilmu Komputer, Universitas Brawijaya

01

MATERI KE-1

Interface Segregation Principle

Bagian 1 dari 3

© Fakultas Ilmu Komputer, Universitas Brawijaya

Klien Tidak Boleh Dipaksa Bergantung pada yang Tidak Dipakainya

ISP adalah SRP yang diterapkan pada antarmuka, bukan pada kelas

Perumusan

- “Clients should not be forced to depend on methods they do not use.”
- **Antarmuka gemuk.** Satu antarmuka besar yang memuat operasi untuk banyak jenis klien sekaligus.
- **Yang dianjurkan.** Beberapa antarmuka kecil, masing-masing sesuai kebutuhan satu jenis klien.

Analogi

- **Alat kendali dengan lima puluh tombol.** Padahal pemakainya hanya perlu tombol daya dan pengatur suara.
- **Akibatnya.** Pemakai dibebani hal yang tidak relevan, dan setiap penambahan tombol mengganggu semua orang.
- **Perbaikannya.** Sediakan kendali yang sesuai kebutuhan tiap jenis pemakai.

Intinya: ukuran antarmuka yang tepat ditentukan oleh kebutuhan kliennya, bukan oleh kelengkapan penyedianya.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Gejala Pelanggaran ISP

Gejala kedua sama dengan gejala pelanggaran LSP — keduanya memang berkerabat

1 Metode kosong pada implementasi

Kelas terpaksa menuliskan badan metode yang tidak berisi apa pun.

2 Pengecualian tidak didukung

Implementasi melempar UnsupportedOperationException.

3 Antarmuka terus membengkak

Setiap kebutuhan baru ditambahkan ke antarmuka yang sudah ada.

4 Kompilasi ulang menular

Menambah satu operasi memaksa seluruh implementasi ikut diperbaiki.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kasus: Antarmuka Perangkat Cetak

Kelas yang sama, dua rancangan antarmuka yang berbeda

Antarmuka

```
interface MesinKantor {
    void cetak(Dokumen d);
    void pindai(Dokumen d);
    void faks(Dokumen d);
}

class PencetakSederhana
        implements MesinKantor {
    public void cetak(Dokumen d) { ... }

    public void pindai(Dokumen d) {
        throw new UnsupportedOperationException();
    }
    public void faks(Dokumen d) { }   // kosong
}
```

Antarmuka terpisah

```
interface Pencetak { void cetak(Dokumen d); }
interface Pemindai { void pindai(Dokumen d); }
interface PengirimFaks { void faks(Dokumen d); }

class PencetakSederhana implements Pencetak {
    public void cetak(Dokumen d) { ... }
}

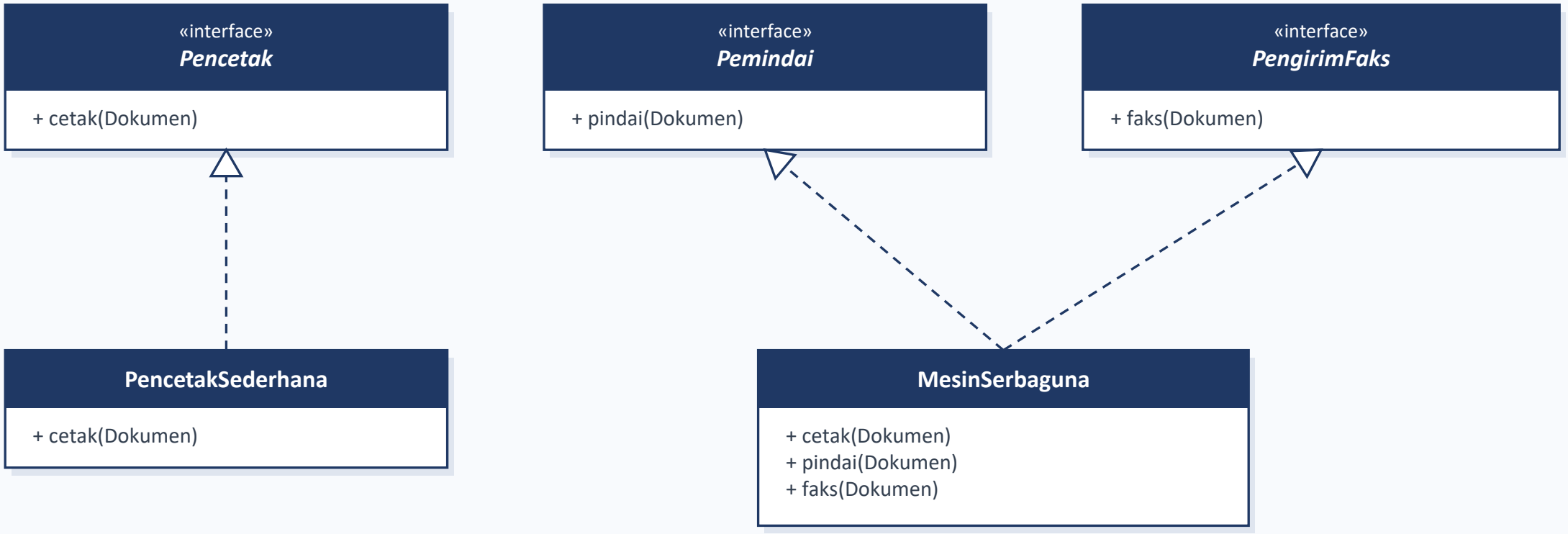
class MesinSerbaguna
        implements Pencetak, Pemindai, PengirimFaks {
    public void cetak(Dokumen d)  { ... }
    public void pindai(Dokumen d) { ... }
    public void faks(Dokumen d)   { ... }
}
```

- **Setelah dipisah, tidak ada satu pun metode kosong dan klien hanya mengenal yang dipakainya.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Struktur Setelah Antarmuka Dipisahkan

Satu kelas boleh memenuhi beberapa antarmuka sekaligus



Garis putus berujung segitiga kosong menyatakan realisasi. MesinSerbaguna juga memenuhi Pencetak.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Dua Kasus Nyata di Lingkungan Akademik

Perhatikan bahwa perbaikan kedua kasus mengecilkan antarmuka, bukan memperbanyak metode

Pemberitahuan kepada mahasiswa

- **Melanggar.** Antarmuka Pemberitahuan memuat kirimSurel(), kirimSms(), dan kirimPesanInstan().
- **Akibatnya.** PemberitahuanSurel mengisi dua metode lain dengan badan kosong.
- **Perbaikan.** Pisahkan menjadi tiga antarmuka; tiap kelas memenuhi yang relevan saja.

Pembayaran uang kuliah

- **Melanggar.** Antarmuka Pembayaran memuat bayarBank(), bayarDompet(), dan bayarKartu().
- **Akibatnya.** PembayaranDompet wajib menuliskan bayarBank() yang tidak relevan.
- **Perbaikan.** Satu antarmuka MetodePembayaran dengan satu operasi bayar(jumlah).

© Fakultas Ilmu Komputer, Universitas Brawijaya

02

MATERI KE-2

Dependency Inversion Principle

Bagian 2 dari 3

© Fakultas Ilmu Komputer, Universitas Brawijaya

Perumusan Dependency Inversion Principle

Empat kalimat pada perumusan Robert C. Martin

- **Modul tingkat tinggi tidak boleh bergantung pada modul tingkat rendah.** Keduanya harus bergantung pada abstraksi.
- **Abstraksi tidak boleh bergantung pada rincian.** Rincianlah yang harus bergantung pada abstraksi.
- **Modul tingkat tinggi.** Aturan bisnis dan kasus penggunaan — bagian yang menjadi alasan perangkat lunak ini dibuat.
- **Modul tingkat rendah.** Basis data, kerangka kerja, pustaka pihak ketiga, protokol jaringan — bagian yang dapat diganti.
- **Analogi.** Stopkontak adalah abstraksi; rumah tidak perlu dirombak setiap kali membeli peralatan listrik baru.

◆ **Perhatikan: yang dibalik bukan alur pemanggilan, melainkan arah ketergantungan pada waktu kompilasi.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Gejala Pelanggaran DIP

Gejala ketiga adalah yang paling mudah diperiksa

1 Kata kunci new pada aturan bisnis

Kelas layanan menciptakan sendiri objek repositori atau penghubung jaringan.

2 Aturan bisnis mengenal kerangka kerja

Nama pustaka atau kerangka kerja muncul di dalam kode aturan bisnis.

3 Pengujian menuntut basis data

Menguji satu aturan sederhana mengharuskan basis data benar-benar berjalan.

4 Perubahan teknis merambat

Mengganti basis data memaksa kelas aturan bisnis ikut diubah.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Kasus: Layanan Pemesanan

Perhatikan siapa yang menentukan implementasi repositori

Melanggar DIP

```
class LayananPesanan {
    private MySqlPesananRepository repo
            = new MySqlPesananRepository();

    public void buat(Pesanan p) {
        if (p.total() <= 0) return;
        repo.insert(p);
    }
}

// Aturan bisnis mengenal MySQL.
// Ganti ke PostgreSQL -> kelas ini diubah.
// Uji tanpa basis data -> tidak mungkin.
```

Memenuhi

```
interface PesananRepository {
    void simpan(Pesanan p);
}

class LayananPesanan {
    private final PesananRepository repo;

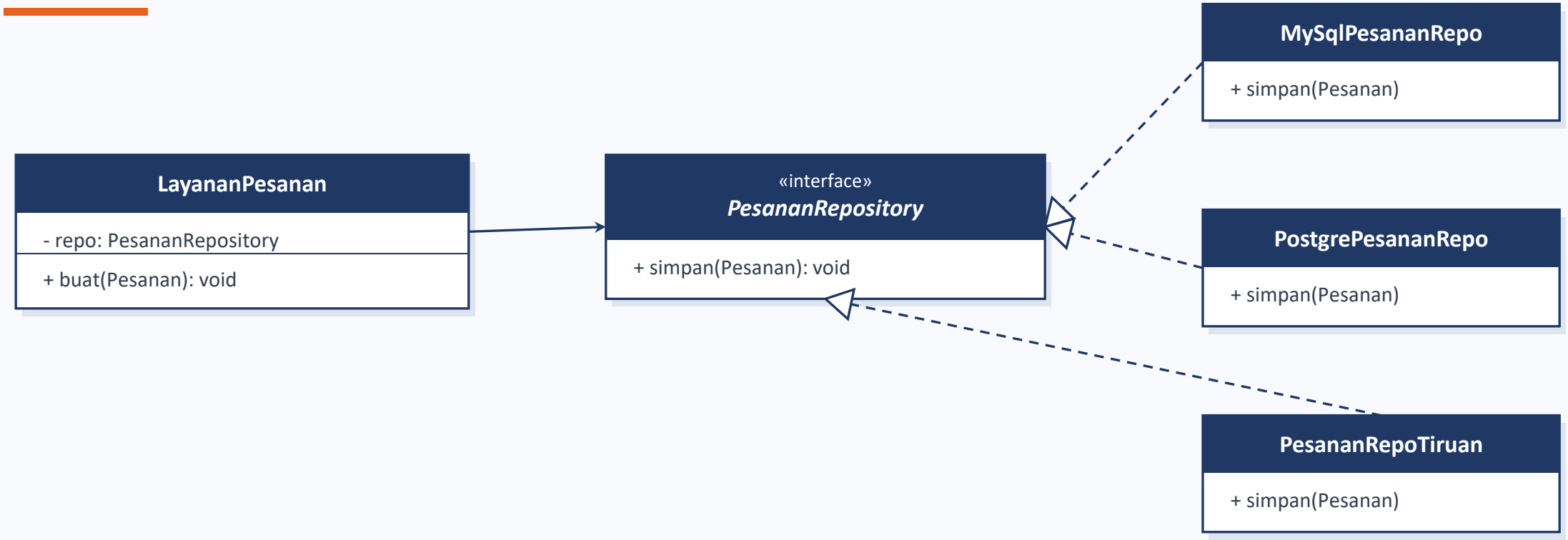
    LayananPesanan(PesananRepository repo) {
        this.repo = repo;
    }
    public void buat(Pesanan p) {
        if (p.total() <= 0) return;
        repo.simpan(p);
    }
}
```

- **Antarmuka PesananRepository dimiliki oleh lapisan aturan bisnis, bukan oleh lapisan basis data.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Arah Ketergantungan Sebelum dan Sesudah Inversi

Panah dari kolom kanan mengarah ke dalam — inilah arti kata “inversion”



- **Sebelum inversi, panah mengarah keluar dari aturan bisnis menuju basis data.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

DIP Membuat Pengujian Menjadi Mungkin

Kemudahan pengujian adalah akibat langsung dari DIP, bukan tujuan tersendiri

LayananPesananTest.java

```java
// Implementasi tiruan cukup memenuhi antarmuka yang sama.
class PesananRepoTiruan implements PesananRepository {
    final List<Pesanan> tersimpan = new ArrayList<>();
    public void simpan(Pesanan p) { tersimpan.add(p); }
}

@Test
void pesananDenganTotalNolTidakDisimpan() {
    PesananRepoTiruan repo = new PesananRepoTiruan();
    LayananPesanan layanan = new LayananPesanan(repo);

    layanan.buat(new Pesanan(0));

    assertTrue(repo.tersimpan.isEmpty());   // tanpa basis data sama sekali
}
```

◆ **Pengujian ini berjalan dalam hitungan milidetik karena tidak menyentuh basis data.**

© Fakultas Ilmu Komputer, Universitas Brawijaya

03

MATERI KE-3

Keterkaitan Kelima Prinsip

Bagian 3 dari 3

© Fakultas Ilmu Komputer, Universitas Brawijaya

SOLID Bekerja sebagai Satu Kesatuan

Kelima prinsip tidak berdiri sendiri-sendiri

1 SRP menyiapkan OCP

Kelas yang hanya punya satu alasan berubah lebih mudah dibuat tertutup bagi modifikasi.

2 LSP menjaga OCP

Turunan yang tidak substitutable memaksa klien memeriksa jenis, sehingga OCP runtuh.

3 ISP menyiapkan LSP

Kontrak yang sempit lebih mudah dipenuhi seluruh implementasinya tanpa metode kosong.

4 DIP mewujudkan OCP

Ketergantungan pada abstraksi adalah cara teknis membuat inti tertutup dan tepi terbuka.

5 OCP adalah tujuannya

Empat prinsip lain pada akhirnya melayani satu hal: perubahan tanpa merusak yang sudah ada.

6 Semua bermuara pada abstraksi

Tanpa abstraksi, tidak satu pun dari kelima prinsip ini dapat dipenuhi.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Peta Ringkas Kelima Prinsip

Kolom terakhir adalah peta jalan menuju pekan ke-7 sampai ke-13

Prinsip	Pertanyaan penguji	Gejala pelanggaran	Pola GoF yang menjawab
SRP	Berapa aktor yang dilayani kelas ini?	Kelas berubah karena banyak permintaan berbeda	Facade, Command
OCP	Apakah fitur baru menuntut kode lama diubah?	Percabangan panjang dan instanceof	Strategy, Decorator, State
LSP	Dapatkah turunan menggantikan induk?	Metode kosong dan pengecualian tak terduga	Composite, Bridge
ISP	Apakah klien memakai semua operasi?	Implementasi berisi metode kosong	Adapter, Facade
DIP	Siapa yang menciptakan objek yang dipakai?	Kata kunci new pada aturan bisnis	Factory Method, Abstract Factory

© Fakultas Ilmu Komputer, Universitas Brawijaya

Studi Kasus: SOLID pada Rancangan Berlapis

Arsitektur berlapis yang benar adalah penerapan DIP pada tingkat modul

Sebelum — tiga lapis yang saling terikat

- **Lapis antarmuka.** Kelas kendali memanggil langsung kelas basis data.
- **Lapis aturan.** Aturan akademik tersebar di kelas kendali dan kelas basis data.
- **Lapis data.** Kelas basis data menyusun sendiri format keluaran untuk layar.
- **Akibatnya.** Mengganti basis data menyentuh ketiga lapis sekaligus.

Sesudah — ketergantungan mengarah ke dalam

- **Lapis antarmuka.** Hanya mengenal antarmuka layanan pada lapis aturan.
- **Lapis aturan.** Memiliki antarmuka repositori, tidak mengenal implementasinya (DIP).
- **Lapis data.** Memenuhi antarmuka repositori milik lapis aturan.
- **Akibatnya.** Mengganti basis data hanya menyentuh satu kelas di lapis terluar.

Intinya: pertanyaan penentunya: siapa yang memiliki antarmuka — pemakai atau penyedianya?

© Fakultas Ilmu Komputer, Universitas Brawijaya

Latihan di Kelas

Dua puluh menit berpasangan — hasilnya menjadi latihan pemanasan untuk Tugas 2

1 Analisis satu berkas

Diberikan satu kelas Java sepanjang kira-kira delapan puluh baris.

Tugas: temukan pelanggaran kelima prinsip beserta nomor barisnya.

2 Susun tabel temuan

Kolom: baris, prinsip yang dilanggar, alasan, usulan perbaikan.

Tugas: format ini sama dengan yang diminta pada LK02.

3 Sketsa rancangan akhir

Gambar class diagram sesudah perbaikan.

Tugas: tunjukkan arah ketergantungan setelah inversi.

© Fakultas Ilmu Komputer, Universitas Brawijaya

Simpulan

- ISP menuntut antarmuka dirancang menurut kebutuhan kliennya, sehingga tidak ada implementasi yang berisi metode kosong.
- DIP menuntut aturan bisnis bergantung pada abstraksi yang dimilikinya sendiri, bukan pada rincian teknis yang dapat diganti.
- Antarmuka pada DIP dimiliki oleh pemakainya, bukan oleh penyedianya — inilah yang membalik arah ketergantungan.
- Kelima prinsip SOLID saling menopang dan seluruhnya bermuara pada satu kemampuan: menambah perilaku tanpa merusak yang sudah ada.
- Rancangan berlapis yang benar adalah penerapan DIP pada tingkat modul, dengan ketergantungan yang selalu mengarah ke dalam.

◆ **Pekan depan: code smell, anti-pattern, dan teknik refactoring — sekaligus penjelasan Tugas 2 (LK02).**

© Fakultas Ilmu Komputer, Universitas Brawijaya

Pustaka

Gaya pengacuan: Harvard-Anglia

- Martin, R.C., 2002. Agile Software Development: Principles, Patterns, and Practices. Upper Saddle River, NJ: Prentice Hall, Bab 11–12.
- Refactoring Guru, 2024. Design Patterns. [daring] Tersedia pada: <https://refactoring.guru/design-patterns> [Diakses 5 Juli 2024].
- Gamma, E., Helm, R., Johnson, R. dan Vlissides, J., 1994. Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA: Addison-Wesley.
- Fowler, M., 2018. Refactoring: Improving the Design of Existing Code. Edisi ke-2. Boston, MA: Addison-Wesley.

© Fakultas Ilmu Komputer, Universitas Brawijaya